

Oi gente, bem vindos de volta, eu me chamo Alexander e hoje darei a aula de introdução ao multiplayer no Godot!

Parte 1- Conceito.

Em um jogo multiplayer, cada jogador vai rodar o mesmo jogo, com os mesmos personagens, objetos e cenas, cada um no seu próprio computador. Mas para ser um jogo realmente multiplayer temos que sincronizar esses objetos, para dar a sensação aos jogadores de que estão em um mesmo ambiente compartilhado.

Imagine uma sala com uma mesa, duas cadeiras e um bloco no centro. Duas pessoas entram na mesma sala e tem a opção de fazer esse bloco pular.

EXEMPLO: Aqui no jogo temos a mesma situação, todos os jogadores tem acesso a este cubo e todos podem apertar barra de espaço para fazê-lo pular, quando apertado espaço estando nessa janela bloco pula, mas na outra mão. Isso acontece por que embora pareça, os dois não são o mesmo bloco, mas sim o mesmo modelo de bloco que está rodando em dois programas separados. Mas tudo o que temos que fazer é sincronizar os dois blocos, fazendo com que toda vez que um pule, o outro também pule ao mesmo tempo, dando a ilusão de que são ambos o mesmo bloco!

Parte 2- Network

Para sincronizar os programas precisamos de um meio de enviar informações de um computador para outro. Se meu personagem soltar uma magia eu quero que essa informação chegue nos outros computadores para que ele solte essa mesma magia no jogo deles.

Há duas maneiras de fazer isso, a primeira é com um servidor dedicado. Um servidor é um computador que vai armazenar dados e gerenciar tudo, quando alguém quer fazer algo como atirar, ele vai mandar um pedido pro servidor que então vai criar uma bala, mandar todos os computadores criarem a bala, e copiarem a posição do servidor. Isso garante mais estabilidade, já que se um jogador cair a partida pode continuar normalmente e é mais difícil de cheatar/trapacear, já que o servidor é a autoridade. Mas servidores costumam a ser caros e de difícil manutenção.

Assim vamos a segunda opção, o peer-to-peer, onde não há um servidor e em vez disso os computadores se comunicam diretamente entre si, normalmente com um computador agindo tanto como servidor como jogador. O que é ótimo pois é grátis, e mais simples de implementar, mas faz com que os jogadores fiquem à mercê da internet do host. Que se cair todo mundo cai.

Pra esse tutorial a gente vai usar o método peer2peer.

Parte 3- Conexão.

No jogo do nosso querido amigo Cubôncio (zoom nele produção), eu implementei esse simples menu e vou explicar o básico do código que conecta tudo pela API multiplayer do godot.

```
var multiplayer_peer = ENetMultiplayerPeer.new()
```

Aqui estamos criando o “comunicador” entre aspas, a gente usa esse objeto como meio para enviar os dados que queremos.

A partir daqui podemos fazer duas coisas:

```
create-server( PORT );
```

```
create-client( ENDEREÇO IP, PORT );
```

Criar um servidor, usando como parâmetro uma porta ou PORT.

Ou criar um cliente, que se conecta a um servidor pelo IP do host e usando a mesma porta.

Ao criar um servidor ele automaticamente usa o próprio IP, pense nele como o endereço de uma casa. Queremos que nossos dados cheguem nessa casa que no caso seria o roteador, e a porta seria a pessoa da casa que queremos que nossos dados cheguem, nesse caso seria o computador que queremos que receba esses dados.

Assim, criando o cliente ele tenta conectar no servidor usando o endereço e a porta, assim podendo enviar e receber dados por esse caminho.

```
multiplayer.multiplayer_peer = multiplayer_peer
```

Depois associamos o nosso peer ao peer a API multiplayer do godot, basicamente damos nosso comunicador para o gerenciador do godot e falamos “Ele tá pronto, agora o conecte na rede!”.

Parte 4 - Sincronia

Agora como sincronizar o cubo? As principais maneiras de compartilhar dados na rede são pelos nodes MultiplayerSynchronizer, MultiplayerSpawner e por RPC's ou Remote Procedure Calls.

Com o MultiplayerSynchronizer, o node vai monitorar as propriedades que você definir, e garantir que elas sejam iguais ao da autoridade do node, ou seja, o que o dono ver os outros também vão ver. Por padrão, as autoridades de todos os nodes são o host/servidor que tem ID 1.

Colocando o node no nosso bloco e falando pra ele monitorar a posição do node, vemos que ao clicar espaço na primeira janela, o bloco pula nas duas janelas, entretanto ao clicar na segunda janela dá pra ver que o bloco dá uma bugada e não pula. Isso por que o node só manda informação da autoridade para os outros clientes e não vice versa, então enquanto o host simplesmente não recebe a info da posição do bloco no computador do

cliente, a info da posição do bloco no host é forçada no computador do cliente, dando esse aspecto bugado onde o bloco chega a se mover mas volta pra posição onde está o bloco da autoridade. Em muitos casos não é um problema porque normalmente não queremos que os jogadores controlem um mesmo personagem, se tivermos dois blocos por exemplo podemos dar autoridade de um dos blocos para o cliente controlar e impedir que um controle o outro.

Esse node é ótimo para compartilhar dados continuamente e por isso é utilizado para sincronizar coisas como posição e HP, que podem mudar o tempo todo.

Mas às vezes queremos rodar algo apenas uma vez de maneira síncrona, como atirar, ou tocar um emote. Para isso usamos as RPC's, elas são funções que podem ser chamadas por um computador para rodar em todos os conectados. Você transforma uma função em rpc escrevendo @rpc em cima. Podemos transformar a ação de pular do Cubôncio em uma rpc

Cada RPC tem 4 configurações:

Modo- Decide quem pode chamar essa função, quando em "authority", apenas a autoridade pode chamar.

-Quando em autoridade o Cubôncio só pula quando o host chama a rpc.

O outro modo "any_peer" deixa que qualquer um chamar a rpc.

-Agora posso fazer o Cubôncio pular pela janela do cliente, chamando a rpc por aqui.

Sync- Decide se a rpc vai rodar no computador que chamou a rpc, quando em call remote, a rpc só vai rodar nos outros computadores.

-Agora se eu chamo a função nessa janela apertando espaço, só a outra vai rodar.

E call_local, que faz com que rode no computador que chamou a função também, útil quando o servidor também é um player.

Modo de transferência, determina como os dados serão transmitidos

"Unreliable"

É o modo mais rápido possível. O computador envia o pacote e "esquece". Ele não verifica se o pacote chegou ao destino nem se os pacotes chegaram na ordem certa.

"Unreliable Ordered"

É o meio-termo, (não garante a entrega), mas garante que os pacotes sejam processados na **sequência correta**.

"Reliable"

Ele garante que **todos** os pacotes cheguem e na ordem exata.

- **Vantagem:** Integridade total dos dados.
- **Quando usar:** Ações críticas que **precisam** acontecer, como **abrir uma porta, morrer, chat de texto ou trocar de fase**. Se o comando "morrer" se perder no modo Unreliable, o jogador continuaria vivo no cliente, gerando um erro de sincronia grave.

Por fim o Canal de Transferência, transferências em reliable só mantém a ordem com outros dados do mesmo canal. ao contrario dos outros três a ordem dele importa, sendo ele o quarto parametro

Se você não especificar a função vai usar os parâmetros, `authority`, `remote_call`, `unreliable` e `0`.

Parte 5- Aplicação + Multiplayer Spawner

No fim fizemos um jogo multiplayer onde os jogadores podem fazer um cubo pular, mas agora podemos tentar fazer algo mais complexo. Vamos fazer um jogo onde os jogadores controlam um quadrado cada, e eles spawnam quando entram.

Para isso a gente pode usar o node Multiplayer Spawner, basicamente se algo nasce no Servidor, ele nasce automaticamente em todos os Clientes. Mas assim como o node do MultiplayerSynchronizer se um cliente instanciar um node ele não vai nascer nos resto.

Então ao clicar host ele spawna um stickman vermelho, e quando eu clico join, ele spawna um azul no servidor e é replicado no cliente. ele também spawna o vermelho que ja estava la antes.

o multiplayer do godot solta sinais quando alguém conecta e desconecta

1. Sinais que o Servidor e Clientes recebem

Estes são disparados para todos os envolvidos na partida quando a estrutura da rede muda.

- **`peer_connected(id: int)`**: Disparado sempre que um **novο** jogador se conecta com sucesso.
 - *Uso comum*: Criar o personagem do novo jogador na tela.
- **`peer_disconnected(id: int)`**: Disparado quando um jogador sai ou perde a conexão.
 - *Uso comum*: Deletar o personagem daquele ID e limpar os dados dele do servidor.

2. Sinais Exclusivos do Cliente

Estes sinais só fazem sentido para quem está tentando se conectar a um servidor.

- **connected_to_server**: Disparado no cliente quando ele consegue estabelecer a conexão com o servidor.
 - *Uso comum*: Mudar a tela de "Carregando" para a tela de "Seleção de Personagem".
- **connection_failed**: Disparado se o cliente tentar conectar, mas o servidor estiver offline ou a porta estiver fechada.
 - *Uso comum*: Mostrar uma mensagem de erro "Não foi possível conectar".
- **server_disconnected**: Disparado quando o servidor "cai" ou fecha a sala.
 - *Uso comum*: Chutar o jogador de volta para o menu principal.

usamos o `multiplayer.peer_connected` e conectamos na função `add player`, que cria o jogador com o id como nome do node, e no jogador colocamos a autoridade do boneco como o seu nome, que é o id do jogador que queremos que o controle.

[FINAL MOSTRANDO O EXEMPLO 2]